case studies

AI-native UX portfolio — five concepts showing the conceptual development, design process, and AI-enhanced process framework behind each one

michaelthibodea(UX)
michaelthbd@yahoo.com · 646 436 2876

Contents

Five AI-native case study concepts, in portfolio order

01	Meridian Research Agent	Real, shipped v1
02	Meridian Exception Manager	Real, shipped v1
03	AI Pattern Library	Synthesis case study
04	Compliance Co-Pilot	Concept / speculative
05	Connected Vehicle Assistant	Concept / speculative

Real, shipped v1 · Anchorpoint Capital / Meridian

Meridian Research Agent

A conversational research agent for investment operations

Analysts used to manually cross-reference five-plus internal systems to investigate a discrepancy. The agent now takes a plain-language question, retrieves from connected sources, and returns a cited, confidence-scored answer.

“I'd have four systems open just to answer one question — and I still wasn't sure I had the full picture.”

illustrative — not a verbatim quote

What's Meridian Research Agent?

Analysts used to manually cross-reference five-plus internal systems to investigate a discrepancy. The agent now takes a plain-language question, retrieves from connected sources, and returns a cited, confidence-scored answer.

Target audience

Investment operations analyst

The problems

- Manually opening 4-5 systems for a single research question
- No consistent starting point — every analyst improvises their own process
- No way to show your work after the fact — answers aren't auditable
- Repetitive, low-judgment lookups take the same time as genuinely hard investigations

Why does this matter?

~45 min

saved per investigation workflow (real, sourced)

Discovery methods: interviews with investment operations analysts across experience levels, plus their team lead; manual workflow mapped step by step

Problem statement: analysts manually pull data from multiple internal systems to answer a research question — slow, repetitive, and inconsistent in quality depending on who does it

Pain points from interviews

1. Manually opening 4-5 systems for a single research question
2. No consistent starting point across analysts
3. No way to show your work after the fact — answers aren't auditable
4. Repetitive lookups take as long as genuinely hard investigations

“If I can't show where an answer came from, I can't actually use it in a regulated environment.”

illustrative — not a verbatim quote

Persona

Investment Operations Analyst

Mid-to-senior level, Anchorpoint Capital

“Detail-oriented and default-skeptical of anything I can't verify.”

Responsibilities

Investigating discrepancies, answering ad hoc research questions, escalating findings

Key challenges

Assembling context across systems with no single source of truth; producing auditable answers

Wishlist

A verifiable answer, faster, without sacrificing traceability

Grounded in a real discovery finding, per the discovery doc — this became the central design constraint.

Pain Points + Business Goals + UX Strategy = Feature Set

Pain Points

- 4-5 systems per question
- No consistent process
- Answers aren't auditable
- Repetitive = hard-investigation time

Business Goals

- Reduce time-to-resolution
- Consistent, trustworthy answers

UX Strategy

- Show sources by default
- Signal confidence always
- Explicit human-gate boundary

Feature Set

- Plain-language query input
- Cited answer format
- Confidence signal
- Show-your-work expand, fallback state

The 8-Stage AI-Enhanced UX Process

Discovery & Research — Real interviews with analysts and team lead. New: mapped what data sources could answer with confidence vs. requiring human judgment.

Problem Definition — Synthesized the four pain points. New: separated ‘slow workflow‘ from ‘unverifiable output’ — only the second needs interaction design, not just a feature.

Competitive / Pattern Analysis — Benchmarked adjacent AI research tools for how they handle uncertainty. Set the bar: ‘the citation is part of the answer,’ not optional.

Design Thinking & Ideation — Centered on agent behavior before any screen: uncertainty, correction, and the pull-vs-handoff line.

Design Sprints & Iteration — Real, iterative sprints with weekly usability testing.

Usability Testing — Tested happy and failure paths equally; watched whether analysts verified sources unprompted.

Delivered Solution — Real, shipped v1 — query input, cited answer, confidence signal, show-your-work expand, fallback state.

Outcomes — See Results — one real sourced metric, three illustrative estimates.

> **What we learned**
> Early versions surfaced confidence only when the agent flagged low confidence itself. Testing showed analysts trusted the agent more — not less — when confidence was shown on every answer.

[VISUAL : Answer View screen]

Results

~45 min
saved per workflow (real)

~82%
answer acceptance (est.)

~12%
escalation rate (est., healthy signal)

~6%
repeat-question rate (est.)

The ~45-minute figure is real and sourced (Michael's resume). The other three are illustrative estimates — no sourced number exists for them.

“The core design challenge wasn't making the agent accurate — it was getting a skeptical, detail-oriented analyst to actually act on an AI answer. Citations weren't an add-on; the citation was part of the answer.“

[VISUAL: Source Detail Expand screen]

Real, shipped v1 · Anchorpoint Capital / Meridian

Meridian Exception Manager

From a flat queue to a ranked worklist the team trusts

An AI layer that detects, prioritizes, and proposes remediation for operational exceptions inside Meridian — presenting them as a ranked, actionable worklist instead of a flat queue.

“Every exception looked the same until I opened it. I had no way to know what actually mattered.”
illustrative — not a verbatim quote

What's Meridian Exception Manager?

An AI layer that detects, prioritizes, and proposes remediation for operational exceptions inside Meridian — presenting them as a ranked, actionable worklist instead of a flat queue.

Target audience

Operations analyst (triage mode) · Team lead

The problems

- Exceptions arrived as a flat queue with no reliable signal for what mattered
- Prioritization depended on tribal knowledge held by experienced analysts
- High-risk exceptions could sit unworked for hours, indistinguishable from routine ones

Why does this matter?

~35%

reduction in exception time-to-resolution
(illustrative target)

Discovery methods: contextual interviews with operations analysts doing manual triage, plus their team lead, capturing the implicit rules experienced staff already use

Problem statement: operational exceptions surfaced through queues and manual triage, so prioritization depended on tribal knowledge of ‘which ones actually matter’

Pain points from interviews

1. Flat queue with no reliable signal for what mattered
2. Prioritization was tribal knowledge — only experienced analysts triaged quickly
3. High-risk exceptions could sit unworked for hours
4. No visibility into why an exception was flagged — re-verified from scratch every time
5. Team leads couldn't confirm prioritization was consistent across the team

“I'd end up re-checking the same thing three different ways just to convince myself it was actually urgent.”

illustrative — not a verbatim quote

Persona

Operations Analyst (Triage Mode)

Same persona as Epic 1, reactive/triage mode

“Tell me why this matters before you ask me to act on it, and I'll trust the ranking.”

Responsibilities

Triaging the exception queue, resolving or escalating flagged breaks

Key challenges

Alert fatigue from over-flagging; no visible reasoning behind prioritization

Wishlist

A ranked worklist, a visible reason on every flag, an easy ‘not an issue’ path that changes future ranking

Reasonably inferred from brief WHO + WHAT + pain points, not sourced persona interviews.

Pain Points + Business Goals + UX Strategy = Feature Set

Pain Points

- Flat queue, no signal for what matters
- Prioritization is tribal knowledge
- No visibility into why flagged
- Aged high-risk exceptions sit unworked

Business Goals

- Faster exception resolution
- Fewer aged/unresolved exceptions
- Analysts trust the prioritization enough to act

UX Strategy

- Contextual interviews to capture implicit rules
- Visible reasoning for every flag by default
- Low-friction dismissal that feeds back

Feature Set

- Prioritized worklist (ranked, not flat)
- Proposed-remediation card
- AI-proposed vs. human-confirmed states
- Aging/escalation indicator

The 8-Stage AI-Enhanced UX Process

Discovery & Research — Contextual interviews with the triage team. New: mapped what the system could rank with confidence vs. needing human judgment, and the trust baseline toward being told vs. asked.

Problem Definition — Synthesized triage pain points. New: separated ‘unranked queue’ from ‘I don't trust being interrupted by a flag.’

Competitive / Pattern Analysis — Benchmarked proactive alerting tools on how they handle being wrong when pushing something — unexplained alerts get ignored within weeks.

Design Thinking & Ideation — Centered on the moment the system interrupts someone: low-confidence behavior, post-dismissal state, detect-vs-confirm boundary.

Design Sprints & Iteration — Earliest version ranked with a bare numeric score; testing showed distrust as ‘another black box’ — replaced with a plain-language reason.

Usability Testing — Tested whether analysts worked the list top-to-bottom (trust) or re-sorted it themselves (distrust signal); tested over-flagging deliberately.

Delivered Solution — Shipped v1 — continuous detection and ranking, visible reason + proposed remediation, AI-proposed vs. human-confirmed states, first-class dismissal.

Outcomes — See Results — illustrative targets; no sourced measured figure exists for this concept.

Unexpected challenge

The riskiest early choice wasn't the ranking algorithm — it was a bare priority score with no reason. Testing showed an unexplained pushed number is trusted less than a flat, unranked queue.

[VISUAL : Exception Worklist screen]

Results (illustrative targets, not measured)

~35%

reduction in avg. time-to-resolution (target)

~40%

reduction in aged/unresolved exceptions (target)

↓

manual re-prioritization rate (trust target)

↓

dismissal-recurrence rate (feedback target)

All figures are illustrative targets — the underlying work is real, but no production measurement exists in this project's files.

“Epic 1 has to earn trust when asked a question; Epic 2 has to earn trust while interrupting someone who didn't ask for anything. That's why alert fatigue and false-positive distrust — not accuracy alone — were the core design challenge.”

[VISUAL: Proposed Remediation Card screen]

Synthesis — grounded in real design-system leadership, constructed narrative

AI Pattern Library

A design system for AI states

A defined set of reusable AI-interaction components — confidence, sourcing, in-progress response, correction, human handoff — documented and governed the same way as any other design-system component.

“We built confidence three different ways across three teams, and none of them talk to each other.”
illustrative, constructed for this case study

What's AI Pattern Library?

A defined set of reusable AI-interaction components — confidence, sourcing, in-progress response, correction, human handoff — documented and governed the same way as any other design-system component.

Target audience

Product designers across multiple teams shipping AI features

The problems

- Every team invents its own confidence visualization, no shared language
- Citation/sourcing shown inconsistently across teams
- No shared 'still generating' affordance — finished or stalled?

Led the design process, drawing on real design-system governance leadership at Wells Fargo, Credit Suisse, and BlackRock; this

Why does this matter?

3+

independently built confidence treatments found in an internal audit (illustrative)

4

different citation/sourcing patterns found (illustrative)

Discovery methods: organizational audit of shipped AI features; interviews with designers/PMs and governance stakeholders; review of published external AI-design guidance

Problem statement: standard design systems don't have components for AI-specific states — no established pattern for sourcing, confidence, in-progress response, or correction

Pain points from interviews

1. Every team invents its own confidence visualization
2. Citation/sourcing shown inconsistently — default, buried, or absent
3. No shared ‘still generating’ affordance
4. No consistent way to tell the system it's wrong, with visible consequence
5. No consistent human-handoff path

“I added a thumbs-down button because it felt right to have one. I have no idea if anyone's ever looked at what it collects.”

illustrative, constructed for this case study

Persona

Product Designer (Adopting Team)

Shipping an AI feature without AI-specific design-system support

"I don't want to invent a confidence indicator from scratch again — I want to know what's already been solved."

Responsibilities

Shipping an AI feature on a deadline; deciding confidence/sourcing/handoff treatment with no internal guidance

Key challenges

No shared starting point for trust-critical states; risk of inconsistency with the product family

Wishlist

A documented, pre-validated component set with clear 'when NOT to use this' guidance

Reasonably inferred from brief WHO/WHAT + constructed audit narrative, not sourced persona interviews.

Pain Points + Business Goals + UX Strategy = Feature Set

Pain Points

- Every team invents its own confidence viz
- Sourcing shown inconsistently
- No shared ‘still generating’ affordance
- No consistent correction/handoff path

Business Goals

- Consistency and speed across teams
- Predictable experience across products

UX Strategy

- Audit ad hoc AI features for recurring problems
- Define & pilot a minimal component set
- Run the proven governance/adoption playbook

Feature Set

- Confidence / Certainty Indicator
- Citation / Source Card
- Streaming-Response State
- Correction + Human-Handoff Banner

The 8-Stage AI-Enhanced UX Process

Discovery & Research — Organizational audit + stakeholder interviews. New: mapped what's reusable across any AI feature vs. product-specific.

Problem Definition — Synthesized the audit. New: separated ‘pattern doesn't exist’ (design system fixes) from ‘users don't trust AI output’ (only component interaction design fixes).

Competitive / Pattern Analysis — The ‘competitor’ is internal — every team's ad hoc solution — benchmarked against published external AI-design guidance.

Design Thinking & Ideation — Defined each component's behavior independent of any one product's implementation; governance treated as part of ideation.

Design Sprints & Iteration — Same cadence as prior non-AI design-system work, iterating on components whose underlying data is non-deterministic.

Usability Testing — Tested trust/comprehension at the component level AND adoption at the governance level — two layers a normal rollout doesn't need.

Delivered Solution — v1 ships 5 documented components, each with an explicit boundary between what the pattern owns and what stays product-specific.

Outcomes — See Results — targets, not measured results.

What we learned

The confidence indicator's first draft used red/yellow/green — pilot review found red collided with the org's error-color language. Redesigned around a neutral intensity scale plus a plain-text qualifier.

[VISUAL: Confidence Indicator — Specimen Sheet]

Results (targets, not measured)

4/6

AI-shipping teams adopting within two quarters (target)

~70%

reduction in inconsistent AI UI patterns (target)

1-2 days

designer-days saved per AI feature (est.)

5-15%

confidence-override rate once embedded (healthy-use target)

This component set has not been piloted or measured in a real rollout — all figures are targets/estimates.

“The hard part was never designing a confidence indicator or a citation card. The hard part is getting competing product teams to actually adopt a shared pattern instead of shipping their own — and for AI states, that governance problem decides whether a user trusts the answer at all.”

[VISUAL: Governance & Adoption Dashboard]

Concept / speculative · Credit Suisse — extends the real OBI case study

Compliance Co-Pilot

Legible multi-agent compliance review

A concept where a research agent, a policy-check agent, and an escalation agent handle parts of submission review — with every action visible on a timestamped trail.

“I never know why my submission is stuck, or who's actually looking at it.”

illustrative — not a verbatim quote

What’s Compliance Co-Pilot?

A concept where a research agent, a policy-check agent, and an escalation agent handle parts of submission review — with every action visible on a timestamped trail.

Target audience

Risk & Compliance manager · Submitter/Employee

The problems

- Submissions returned for missing information, with no clear explanation
- No visibility into where a submission is in the review pipeline or why
- No consolidated view of a submitter's disclosure history for judgment calls

Why does this matter?

~40%

of OBI submissions returned for missing info
(illustrative baseline)

Discovery methods: real, carried over from the OBI engagement: interviews with Risk & Compliance managers and Submitter/Employee personas

Problem statement: a single submission gets manually routed and reviewed step by step, with no visibility into why it's at a given stage or what's being checked

Pain points from interviews

1. Submissions returned for missing information, with no clear explanation
2. No visibility into where a submission is in the review pipeline or why
3. No consolidated view of a submitter's disclosure history for judgment calls

“It's a black box. I submit, then I wait, and I don't know what's happening in between.”

illustrative — not a verbatim quote

Persona

Risk & Compliance Manager

Primary persona, Credit Suisse OBI workflow

“I need to trust the system's recommendation, not just accept it — show me why.”

Responsibilities

Reviewing submissions for policy conflicts; making judgment calls; escalating

Key challenges

Opacity of automated flags; re-verifying findings with no visible reasoning

Wishlist

A visible reasoning trail; override without losing the audit record

Illustrative sentiment, reasonably inferred from brief WHO/WHAT, not a sourced verbatim.

Pain Points + Business Goals + UX Strategy = Feature Set

Pain Points

- Returned with no explanation
- No pipeline visibility
- No disclosure history view

Business Goals

- Faster, more consistent review with a clear audit trail

UX Strategy

- Extend real OBI findings
- Agent proposes, human confirms
- Visible activity trail

Feature Set

- Research, policy-check & escalation agents
- Activity trail
- Policy linking
- Override + log

The 8-Stage AI-Enhanced UX Process

Discovery & Research — Real interviews carried over from OBI. New: mapped which parts of the workflow are agent-automatable vs. requiring human compliance judgment.

Problem Definition — Same OBI pain points. New: separated slow workflow from opaque-automation distrust.

Competitive / Pattern Analysis — Not yet conducted for this concept — flagged as an open next step.

Design Thinking & Ideation — Centered on agent behavior: what each surfaces, how conflicts are shown, where agent-proposes/human-confirms sits.

Design Sprints & Iteration — Not yet run. Activity trail, policy linking, override+log are proposed, not tested.

Usability Testing — Not yet conducted — concept/vision piece, no fabricated test results.

Delivered Solution — Concept only — agents propose, flag, and surface; a human retains final authority on every material action.

Outcomes — See Results — targets, not measured results.

Unexpected challenge

The hardest open question isn't agent accuracy — it's whether reviewers will trust a workflow where three different agents can each touch one case.

[VISUAL : Submission Detail — Agent Activity Trail]

Results (targets, not measured)

~30%

faster review cycle (target)

~50%

fewer missing-info returns (target)

≥80%

reviewer trust target

This concept has not been piloted or tested — all figures are explicit targets.

“When more than one agent can act on a case, the UX problem shifts from 'is the AI right' to 'can the user tell who did what and why.' The activity trail is the design's direct answer to that question.”

[VISUAL: Policy Conflict Drill-down]

Concept / speculative · American Honda — extends real service-design research

Connected Vehicle Assistant

One coherent owner-facing assistant

A concept where a scheduling agent, a diagnostics agent, and a service-recommendation agent coordinate behind a single owner-facing assistant — with every entry labeled by which agent generated it.

“I get an alert, a reminder, and a text — and none of them talk to each other, so I don't know what actually matters.”

illustrative — not a verbatim quote

What’s Connected Vehicle Assistant?

A concept where a scheduling agent, a diagnostics agent, and a service-recommendation agent coordinate behind a single owner-facing assistant — with every entry labeled by which agent generated it.

Target audience

Vehicle owner using a connected-vehicle mobile app

The problems

- Owners get fragmented, one-off notifications with no unified assistant tying scheduling, diagnostics, and recommendations together

Extends design and discovery work from the real American Honda service-design engagement into this forward-looking multi-agent

Why does this matter?

~1/3

of service-center calls are 'what does this alert mean' (illustrative estimate)

Discovery methods: real, carried over from prior Honda engagement: customer interviews and journey-mapping work

Problem statement: owners get fragmented, one-off notifications with no unified assistant tying scheduling, diagnostics, and service recommendations together

Pain points from interviews

1. Fragmented, one-off notifications with no unified assistant tying scheduling, diagnostics, and recommendations together

“Every alert feels like it's from a different app, even though it's the same car.”

illustrative — not a verbatim quote

Persona

Vehicle Owner

Connected-vehicle mobile app user

“Just tell me plainly what's wrong and what I need to do — I don't want to decode a warning light myself.”

Responsibilities

Keeping up with maintenance; interpreting alerts; deciding when/where to schedule service

Key challenges

Fragmented notifications; uncertainty about alert urgency; friction from alert to appointment

Wishlist

One place for vehicle status; plain-language explanations; one-tap scheduling

Reasonably inferred from brief WHO/WHAT, not sourced persona interviews for this concept.

Pain Points + Business Goals + UX Strategy = Feature Set

Pain Points

- Fragmented, one-off notifications
- No plain-language alert explanation
- Scheduling requires a separate app/flow

Business Goals

- Increase service-scheduling conversion
- Strengthen the owner-dealer relationship

UX Strategy

- Extend real Honda findings
- Agent proposes, owner confirms for cost/scheduling
- Single feed with per-entry labeling

Feature Set

- Diagnostics agent
- Scheduling agent
- Service-recommendation agent
- Unified feed + diagnostic detail + scheduling screen

The 8-Stage AI-Enhanced UX Process

Discovery & Research — Real customer interviews and journey-mapping (carried over from Honda). New: mapped which parts are agent-automatable vs. needing a technician's judgment.

Problem Definition — Synthesized the fragmented-notifications pain point. New: separated disconnected notifications from distrust of a warning light's claim.

Competitive / Pattern Analysis — Not yet conducted for this concept — flagged as an open next step.

Design Thinking & Ideation — Centered on agent behavior: what each surfaces, how related outputs merge vs. pile up, where agent-proposes/owner-confirms sits.

Design Sprints & Iteration — Not yet run. Unified feed, plain-language translation, consolidation, one-tap scheduling are proposed. Surface decided: mobile-only.

Usability Testing — Not yet conducted — concept/vision piece, no fabricated test results.

Delivered Solution — Concept only — agents propose and surface; owner confirmation finalizes anything with scheduling/cost implication.

Outcomes — See Results — targets, not measured results.

Unexpected challenge

The hardest question isn't whether each agent is accurate — it's whether stitching three agents together reduces notification volume, or just relabels the same volume with better attribution.

[VISUAL : Unified Assistant Feed screen]

Results (targets, not measured)

~20%

uplift in scheduling conversion (target)

~35%

reduction in 'what does this alert mean' calls (target)

≥75%

owner-understanding trust target

This concept has not been piloted or tested — all figures are explicit targets.

“When three agents can each independently reach the owner, the question isn't 'does each agent work' — it's 'does the owner end up with one coherent assistant, or three notification sources wearing one label.' Entry consolidation is the concept's actual answer to that pile-up risk.”

[VISUAL: Diagnostic Alert Detail screen]

michaelthibodea(UX)

646 436 2876 · michaelthbd@yahoo.com